

हेलो! आपका स्वागत है **Unit 20: Advanced JavaScript** की complete study class में.

मैंने आपके workspace में, unit 20 adv js folder के अंदर (case matching directories checks) ये complete study files write कर दी हैं:

- Study Guide File: chapter20_advanced_js.md
- Code Examples File: advanced_js_examples.js
- Practice Exercises File: advanced_js_practice.js

चलिए अब Advanced JS programming paradigms, memoization caches, debouncing events throttling setups, shallow copies vs deep copies structures ko simple Hinglish explanation ke sath samajhna shuru karte hain.

PART 1: OPTIMIZATIONS & UTILITIES

Function execution parameters aur variables scopes configurations ko tune aur optimize karne ke methods.

1. Closures Revision

What is it? Closure scope state memory pointers variables ka lookup context map hai.

Nesting function checks: parent function execute hokar close stack memory se hat jata hai tab bhi child variables state safe reference track hold.

Simple example

javascript

```
function init() {  
  let activeKey = "secret123";  
  return () => `Key is: ${activeKey}`;  
}
```

```
let keyObj = init();
```

```
console.log(keyObj());
```

Output Key is: secret123

2. Currying

What is it? Currying transforms functions calling sequences from standard $f(a, b)$ format to chain format $f(a)(b)$.

Simple example

javascript

```
const add = a => b => a + b;
```

```
console.log(add(5)(10));
```

Output 15

3. Memoization

What is it? Memoization matches calculated function args with results cached inside local memory closure tables, returning values directly for repeated inputs.

Simple example

javascript

```
function makeMemo() {  
  let cacheObj = {};  
  return function(num) {  
    if (num in cacheObj) return cacheObj[num];  
    let computedVal = num * num;  
    cacheObj[num] = computedVal;  
    return computedVal;  
  };  
}
```

```
let memoSquare = makeMemo();
```

```
console.log(memoSquare(10));
```

4. Debouncing & Throttling

What is it?

- **Debouncing:** delays operational callbacks execution till user ceases interactions for specified time intervals (e.g. typing searches).
- **Throttling:** executes target handler strictly only once per constant time intervals (e.g. window scrolling updates).

PART 2: MEMORY MANAGEMENT & COPY OPERATIONS

1. Value vs Reference

- **Primitive types (Value):** copied by value content. Modifying one copy does not affect the original.
- **Objects/Arrays (Reference):** copied by memory location pointer address references. Edits on copy mutates original structures instantly.

Simple example

javascript

```
let val1 = 10;
```

```
let val2 = val1; // Copied by value
```

```
val2 = 20;
```

```
console.log("val1:", val1);
```

```
let profile1 = { name: "Ravi" };
```

```
let profile2 = profile1; // Copied by reference
```

```
profile2.name = "Amit";
```

```
console.log("profile1 name:", profile1.name);
```

Output val1: 10 profile1 name: Amit

2. Shallow Copy vs Deep Copy

- **Shallow Copy:** copies only root level keys properties. Nested child objects references continue to point to original memory cells (Spread operator handles shallow clones).
- **Deep Copy:** recursively duplicates all nested layer objects, creating isolated standalone clones (structuredClone() or JSON stringify/parse).

Simple example

javascript

```
let base = { id: 1, info: { tag: "JS" } };
```

```
// Shallow
```

```
let shallow = { ...base };
```

```
shallow.info.tag = "Web"; // mutates base.info.tag!  
  
// Deep  
  
let deep = JSON.parse(JSON.stringify(base));  
  
deep.info.tag = "Advanced"; // base.info.tag remains unaffected!
```

3. Garbage Collection

Unused memory references checks: JS engine automatically runs "Mark-and-Sweep" tracking variable scopes references reachable links. Disconnected variables values are reclaimed. Developers do not write manual `free()` operations.

PART 3: PURE ACTIONS & FP PARADIGMS

1. Pure Functions & Immutability

- **Pure Function:** same input arguments consistently return identical outputs without side effects.
- **Immutability:** states details are never updated in-place. Updates create fresh state clones, protecting historical logs.

Simple example

```
javascript  
  
// Pure  
  
const multiply = (x, y) => x * y;  
  
// Immutability update  
  
let list = [1, 2];  
  
let updatedList = [...list, 3]; // fresh array clone
```

QUICK REVISION SUMMARY

- **Closures:** Nested lexical function context mappings.
- **Currying:** Sequential single argument execution setups.
- **Memoization:** Speeds up operations using closure caches.

- **Debounce**: waits for typing pause silence; Throttling runs periodically at constant interval cycles.
 - **Clones**: Shallow copies first levels; Deep copies all nested structures.
 - **Value vs Reference**: Primitives copy values; Objects copy reference pointers.
 - **FP basics**: Coding design prioritizing Pure functions and Immutability.
-



IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **Memoization Cache**: Operational dictionary storing pre-computed results mapped to unique function inputs.
 2. **Debounce Pause**: Timeout period mapping trigger delays to the cessation of input interactions.
 3. **Throttling Constant**: Duration specifying standard execution intervals for recurring events.
 4. **Structured Clone**: Native JS browser constructor resolving deep copy conversions.
 5. **Pure Function**: Mathematical function structure executing consistently without mutable side-effects.
-



IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. Difference between Debouncing and Throttling?

Ans:

- Debouncing triggers the function only after a certain period of inactivity (silence delay). Useful for search auto-suggest.
- Throttling limits the execution frequency, running the function at most once every specified time interval during continuous trigger actions. Useful for page scroll tracking.

Q2. Difference between Shallow Copy and Deep Copy?

Ans:

- Shallow copy duplicates the top-level property references of an object; nested objects are still linked by reference.
- Deep copy duplicates all nested structural layers completely, creating a fully independent clone.

Q3. What is a Pure Function?

Ans: A function that consistently returns the same output for the same arguments, and performs its operations without any side effects (like mutating global variables or logging to external APIs).

Q4. Explain currying in JavaScript.

Ans: Currying is the design practice of transforming a function that takes multiple arguments into a sequence of nested functions, each accepting a single argument.

Q5. Predict output:

javascript

```
let arr1 = [1, 2];
```

```
let arr2 = arr1;
```

```
arr2.push(3);
```

```
console.log(arr1);
```

Ans: [1, 2, 3]. (Since arrays are copied by reference, both point to the same memory location).

Q6. How do you implement a simple memoization function?

Ans: By using a closure where a local cache object persists and stores computation keys mapping inputs to output values.

Q7. Predict output:

javascript

```
let original = { a: 1, b: { c: 2 } };
```

```
let copy = { ...original };
```

```
copy.b.c = 99;
```

```
console.log(original.b.c);
```

Ans: 99. (Spread operator creates a shallow copy, leaving nested object references linked).

Q8. What is Immutability in functional programming?

Ans: The paradigm where data variables are never edited in-place; instead, updates create fresh copies, preserving original structures.

Q9. How does Garbage Collection trace unreachable memory?

Ans: JS engine uses the "Mark-and-Sweep" algorithm. It starts from root contexts (like global scope) to tag reachable items. Untagged items are identified as unreachable garbage and deleted.

Q10. Predict output:

javascript

```
console.log(typeof structuredClone);
```

Ans: "function". (In modern browsers/Node.js).

Q11. Predict output:

javascript

```
const add = a => b => c => a + b + c;
```

```
console.log(add(1)(2)(3));
```

Ans: 6. (Curried function structure).

Q12. What does JSON.parse(JSON.stringify(obj)) do?

Ans: Creates a deep copy of obj. Note: It ignores methods (functions), undefined values, and Symbols.

Q13. Predict output:

javascript

```
let x = "Hello";
```

```
let y = x;
```

```
y = "World";
```

```
console.log(x);
```

Ans: "Hello". (Strings are primitive types copied by value).

Q14. What are side effects in function calls?

Ans: Changes a function makes outside its own scope, such as modifying global variables, DOM structures, writing files, or printing console logs.

Q15. Predict output:

javascript

```
let val = Object.freeze({ name: "Amit" });
```

```
val.name = "Ravi";
```

```
console.log(val.name);
```

Ans: "Amit". (Object.freeze prevents property mutations).

Q16. Can we invoke garbage collection manually in production JavaScript?

Ans: No. JavaScript engines manage memory allocation and garbage collection automatically.

Q17. Why is currying useful?

Ans: It facilitates function composition, makes functions reusable through partial application, and helps in configuring base handler parameters.

Q18. Predict output:

```
javascript
```

```
let list = [10, 20];
```

```
let copy = Object.assign([], list);
```

```
copy[0] = 99;
```

```
console.log(list[0]);
```

Ans: 10. (Object.assign creates a shallow copy of arrays; top-level changes do not mutate original).

Q19. What is caching in Memoization?

Ans: An in-memory key-value dictionary that stores inputs and pre-computed outputs to avoid redundant CPU calculations.

Q20. What is declarative programming style?

Ans: Focuses on "what to do" rather than "how to do it" (imperative style), utilizing higher-order collection functions like map, filter, and reduce.



PRACTICE QUESTIONS

Question 1: Simple Curried multiplier

- **Question:** Create a curried function multiplying three numbers: mul(a)(b)(c).
- **Solution:**

```
javascript
```

```
const mul = a => b => c => a * b * c;
```



```
console.log(mul(2)(3)(4)); // 24
```

- **Explanation:** Sequential arrow declarations construct chained single-argument evaluation scopes.

Question 2: Custom Shallow vs Deep copy checks

- **Question:** Construct custom deep copy cloning helper for object { a: 1, info: { tag: "JS" } }.
- **Solution:**

javascript

```
let data = { a: 1, info: { tag: "JS" } };  
  
let cloned = JSON.parse(JSON.stringify(data));  
  
cloned.info.tag = "Advanced";  
  
console.log("Original tag:", data.info.tag); // JS  
  
console.log("Cloned tag:", cloned.info.tag); // Advanced
```

- **Explanation:** Deep copy isolates nested objects, so modifying cloned does not mutate data.

Question 3: Pure function checker

- **Question:** Turn the impure calculation function into a pure function.
- **Impure code:**

javascript

```
let discount = 10;  
  
function applyImpure(price) { return price - discount; }
```

- **Pure solution code:**

javascript

```
function applyPure(price, discountAmount) {  
  
    return price - discountAmount;  
  
}
```

```
console.log(applyPure(100, 10)); // 90
```

- **Explanation:** Passing all dependencies (discountAmount) explicitly as arguments ensures the function is pure, eliminating external mutable states.

